

CUDA C++ for AI/ML Interviews

Beginner to expert roadmap, architecture notes, runnable code, execution commands, and interview drills

Audience: a full-stack engineer moving into C++ CUDA, GPU computing, AI/ML systems, inference, training, and performance engineering.

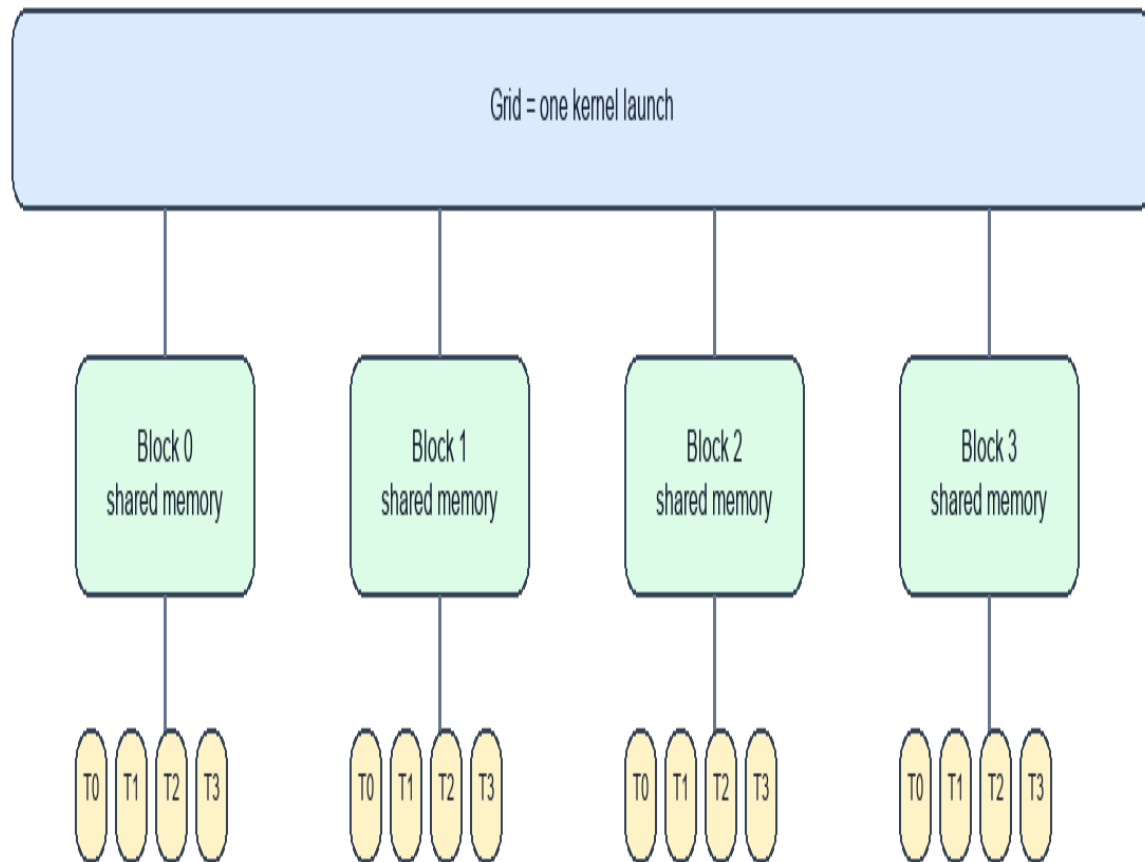
How to use this PDF: read it in passes: first understand the mental model, then compile every sample, then profile and explain each optimization aloud.

0. What CUDA C++ Is

CUDA C++ is C++ plus NVIDIA GPU extensions. You write normal host code that runs on the CPU and kernel code that runs on the GPU. The GPU is a throughput machine designed to run many lightweight threads and hide memory latency with massive concurrency.

- CPU code is host code. GPU code is device code.
- A kernel is a GPU function launched from the CPU with `<<>>` syntax.
- A warp is a hardware scheduling group, commonly 32 threads.
- A block is a group of threads that can synchronize and share fast shared memory.
- A grid is all blocks created by one kernel launch.

CUDA Execution Hierarchy



A kernel launches a grid. A grid has blocks. Blocks have threads. Threads in a block can cooperate through shared memory and barriers.

1. Setup And First Execution

- 1 Install an NVIDIA driver that supports your GPU.
- 2 Install the CUDA Toolkit. On Windows, install a compatible Visual Studio C++ toolchain first.
- 3 Confirm the toolchain with: `nvcc --version`
- 4 Confirm the GPU and driver with: `nvidia-smi`
- 5 Compile a .cu file with: `nvcc file.cu -o file.exe`
- 6 Run the executable from PowerShell: `.file.exe`

```
cd C:\Users\ygirish\Documents\Codex\2026-05-28\hi\cuda_samples
```

```
nvcc 01_hello.cu -o 01_hello.exe
.\01_hello.exe
```

2. CUDA Language Keywords

Keyword	Meaning	Interview signal
__global__	Function runs on GPU and is launched by CPU.	Used for kernels; returns void.
__device__	Function or variable lives on GPU device.	Callable from device code.
__host__	Function runs on CPU host.	Can combine with __device__ for dual-use helpers.
__shared__	Per-block shared memory.	Fast cooperation inside one block.
__syncthreads()	Barrier for threads in a block.	Use before reading shared data written by peers.
threadIdx/blockIdx	Built-in coordinates.	Used to map work to data.
blockDim/gridDim	Launch dimensions.	Used for bounds and grid-stride loops.

3. Memory Model

CUDA Memory Hierarchy



Interview rule: performance usually starts with reducing global-memory traffic and making remaining accesses coalesced.

Memory	Lifetime / scope	Typical use	Pitfall
Registers	One thread	Local scalars	Too many registers can reduce occupancy.
Local memory	One thread, backed by global	Spilled arrays/variables	Slow if compiler spills registers.
Shared memory	One block	Tiling, reductions, stencil reuse	Bank conflicts and missing barriers.

Memory	Lifetime / scope	Typical use	Pitfall
Global memory	Whole device allocation	Large arrays and tensors	Latency, non-coalesced access.
Constant memory	Device-wide read-only	Small broadcast constants	Bad for random per-thread access.
Unified memory	Managed CPU/GPU address	Simplicity and prototyping	Page migration surprises.
Pinned host memory	CPU page-locked	Async copies and overlap	Overuse hurts system memory.

Interview answer: coalescing means adjacent threads in a warp access adjacent memory addresses, allowing the hardware to combine requests efficiently.

4. Writing Correct Kernels

The most common CUDA beginner bug is forgetting that you usually launch more threads than data elements. Every kernel that maps one thread to one element needs a bounds check.

```
__global__ void saxpy(float a, const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a * x[i] + y[i];
}
```

- Always check `cudaGetLastError` after a kernel launch while learning.
- Use `cudaDeviceSynchronize` when you need to catch asynchronous kernel errors immediately.
- Test small sizes, odd sizes, and sizes smaller than one block.
- Compare GPU output against a simple CPU reference before optimizing.

5. Grid-Stride Loops

A grid-stride loop lets each thread process multiple elements. This makes kernels work for arbitrary N and lets you tune launch size independently from input size.

```
__global__ void add(float* x, int n) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < n;
         i += blockDim.x * gridDim.x) {
        x[i] += 1.0f;
    }
}
```

6. Shared Memory And Tiling

Shared memory is a programmer-managed cache scoped to a block. In matrix multiplication, tiling loads submatrices into shared memory so many threads reuse the same global-memory values.

- Each block cooperatively loads a tile from A and a tile from B.
- Threads synchronize after loading tiles.
- Each thread accumulates one output element using tile data.
- The pattern reduces global reads from repeated per-thread loads to reused block-level loads.

```
__shared__ float As[TILE][TILE];
__shared__ float Bs[TILE][TILE];
As[threadIdx.y][threadIdx.x] = A[row * N + tiled_col];
Bs[threadIdx.y][threadIdx.x] = B[tiled_row * N + col];
__syncthreads();
```

7. Synchronization

Tool	Scope	Use
__syncthreads()	One block	Barrier before consuming shared data.
Atomic operations	Address/global or shared	Correctness for concurrent updates; can serialize.
cudaDeviceSynchronize()	Whole device from host	Wait for queued work before CPU reads or exits.
Stream sync	One stream	Wait for ordered work in a stream.
Events	Device timestamp/order	Measure GPU time or create cross-stream dependencies.

Common trap: __syncthreads() only synchronizes threads in the same block. CUDA has no cheap global barrier inside a normal kernel.

8. Performance Model

Streaming Multiprocessor Mental Model

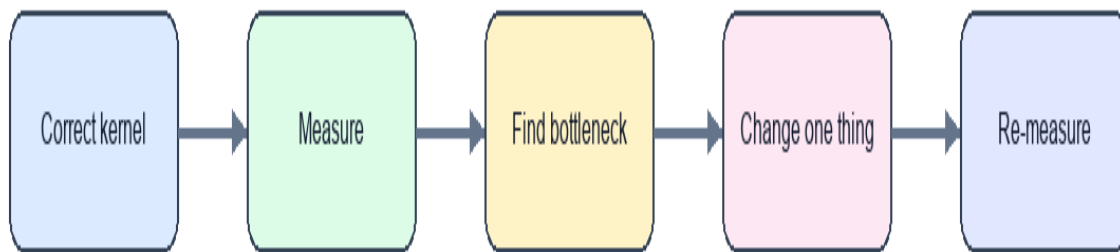


A CUDA performance explanation usually starts with three questions: Is the kernel memory-bound or compute-bound? Is memory access coalesced? Is there enough occupancy and instruction-level work to hide latency?

Concept	Plain-English meaning	How to improve
Occupancy	How many warps can reside on an SM.	Reduce excessive registers/shared memory; choose reasonable block sizes.
Latency hiding	Switch to another ready warp while one waits.	Expose enough parallelism and independent work.
Arithmetic intensity	Math per byte moved.	Reuse data through shared memory, caches, or better algorithms.
Divergence	Threads in a warp take different branches.	Group similar work; simplify branch-heavy kernels.
Coalescing	Warp memory addresses form efficient transactions.	Use structure-of-arrays and contiguous indexing.
Bank conflicts	Shared memory accesses collide by bank.	Pad or reorganize shared arrays.

9. Profiling Workflow

CUDA Optimization Loop



Never optimize from vibes. Use CUDA events for timing, Nsight Systems for timeline, and Nsight Compute for kernel-level evidence.

- Use CPU timers only for end-to-end user-visible latency.
- Use CUDA events for GPU elapsed time between operations.
- Use Nsight Systems to see CPU/GPU overlap, transfers, gaps, and stream timelines.
- Use Nsight Compute to inspect one kernel: occupancy, memory throughput, cache behavior, divergence, and instruction mix.
- Change one thing at a time; keep the CPU reference and correctness tests alive.

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start);
kernel<<<blocks, threads>>>(...);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float ms = 0.0f;
cudaEventElapsedTime(&ms, start, stop);
```

10. Streams, Async Copies, And Overlap

A CUDA stream is an ordered queue of work. Operations in the same stream run in issue order. Independent streams may overlap if the hardware and dependencies allow it.

- Use `cudaMemcpyAsync` plus pinned host memory for asynchronous host-device transfers.
- Use multiple streams to pipeline copy, compute, and copy-back chunks.
- Avoid synchronizing the whole device unless necessary.
- Remember that default-stream behavior matters; know whether your code uses legacy or per-thread default stream semantics.

11. AI/ML CUDA Concepts

Topic	What to know for interviews
Tensor cores	Specialized matrix-multiply hardware used by deep learning kernels.
Mixed precision	FP16/BF16/TF32/FP8 trade precision for throughput and memory savings.
cuBLAS/cuDNN	Use vendor libraries for GEMM, convolutions, reductions, and neural-network primitives.
Custom ops	Write CUDA extensions when framework primitives do not express your operation efficiently.
Memory bandwidth	Large models are often limited by data movement, not raw FLOPs.
Batching	Improves throughput by increasing parallel work and matrix sizes.
Kernel fusion	Combines operations to reduce intermediate memory traffic and launch overhead.

12. Debugging And Correctness

- Compile debug builds with `-G` while debugging, then remove `-G` for performance measurements.
- Use compute-sanitizer to catch illegal memory access, race conditions, and uninitialized values.
- Keep CPU reference implementations for kernels with tricky indexing.
- Check every CUDA API call with a macro during development.
- Treat `printf` inside kernels as a debugging tool only; it changes timing and can be buffered.

```
nvcc -G 02_vector_add.cu -o debug_vector_add.exe
compute-sanitizer .\debug_vector_add.exe
```

13. Runnable Sample Programs

File	Concept	Compile command
01_hello.cu	Kernel launch and thread coordinates	nvcc 01_hello.cu -o 01_hello.exe
02_vector_add.cu	Memory allocation, copies, error checking	nvcc 02_vector_add.cu -o 02_vector_add.exe
03_matrix_mul_tiled.cu	Shared-memory tiling	nvcc 03_matrix_mul_tiled.cu -o 03_matrix_mul_tiled.exe
04_reduction.cu	Parallel reduction	nvcc 04_reduction.cu -o 04_reduction.exe
05_streams_async.cu	Pinned memory and async stream work	nvcc 05_streams_async.cu -o 05_streams_async.exe
06_unified_memory.cu	Managed memory	nvcc 06_unified_memory.cu -o 06_unified_memory.exe

Vector Add Example

```
#include <cuda_runtime.h>
#include <iostream>
#include <vector>

#define CUDA_CHECK(call) do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        std::cerr << "CUDA error: " << cudaGetErrorString(err) \
        << " at " << __FILE__ << ":" << __LINE__ << "\n"; \
        std::exit(1); \
    } \
} while (0)

__global__ void vector_add(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}

int main() {
    int n = 1 << 20;
    size_t bytes = n * sizeof(float);
    std::vector<float> h_a(n, 1.0f), h_b(n, 2.0f), h_c(n);
    float *d_a = nullptr, *d_b = nullptr, *d_c = nullptr;

    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    CUDA_CHECK(cudaMalloc(&d_c, bytes));
    CUDA_CHECK(cudaMemcpy(d_a, h_a.data(), bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b.data(), bytes, cudaMemcpyHostToDevice));

    int threads = 256;
    int blocks = (n + threads - 1) / threads;
    vector_add<<<blocks, threads>>>(d_a, d_b, d_c, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaMemcpy(h_c.data(), d_c, bytes, cudaMemcpyDeviceToHost));

    std::cout << "c[123] = " << h_c[123] << "\n";
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

Tiled Matrix Multiplication Example

```
#include <cuda_runtime.h>
#include <iostream>
#include <vector>

constexpr int TILE = 16;

__global__ void matmul_tiled(const float* A, const float* B, float* C, int N) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;
```

```

for (int t = 0; t < (N + TILE - 1) / TILE; ++t) {
    int a_col = t * TILE + threadIdx.x;
    int b_row = t * TILE + threadIdx.y;
    As[threadIdx.y][threadIdx.x] = (row < N && a_col < N) ? A[row * N + a_col] : 0.0f;
    Bs[threadIdx.y][threadIdx.x] = (b_row < N && col < N) ? B[b_row * N + col] : 0.0f;
    __syncthreads();

    for (int k = 0; k < TILE; ++k) {
        sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
    }
    __syncthreads();
}
if (row < N && col < N) C[row * N + col] = sum;
}

int main() {
    int N = 512;
    size_t bytes = N * N * sizeof(float);
    std::vector<float> A(N * N, 1.0f), B(N * N, 2.0f), C(N * N);
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, bytes); cudaMalloc(&d_B, bytes); cudaMalloc(&d_C, bytes);
    cudaMemcpy(d_A, A.data(), bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B.data(), bytes, cudaMemcpyHostToDevice);
    dim3 threads(TILE, TILE);
    dim3 blocks((N + TILE - 1) / TILE, (N + TILE - 1) / TILE);
    matmul_tiled<<<blocks, threads>>>(&d_A, &d_B, &d_C, N);
    cudaMemcpy(C.data(), d_C, bytes, cudaMemcpyDeviceToHost);
    std::cout << "C[0] = " << C[0] << "\n";
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}

```

Reduction Example

```

#include <cuda_runtime.h>
#include <iostream>
#include <numeric>
#include <vector>

__global__ void reduce_sum(const float* in, float* block_sums, int n) {
    extern __shared__ float s[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x * 2 + threadIdx.x;
    float x = 0.0f;
    if (i < n) x += in[i];
    if (i + blockDim.x < n) x += in[i + blockDim.x];
    s[tid] = x;
    __syncthreads();

    for (unsigned int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) s[tid] += s[tid + stride];
        __syncthreads();
    }
    if (tid == 0) block_sums[blockIdx.x] = s[0];
}

int main() {
    int n = 1 << 20;
    std::vector<float> h(n, 1.0f);
    int threads = 256;
    int blocks = (n + threads * 2 - 1) / (threads * 2);
    float *d_in, *d_sums;
    cudaMalloc(&d_in, n * sizeof(float));
    cudaMalloc(&d_sums, blocks * sizeof(float));
    cudaMemcpy(d_in, h.data(), n * sizeof(float), cudaMemcpyHostToDevice);
    reduce_sum<<<blocks, threads, threads * sizeof(float)>>>(&d_in, &d_sums, n);
    std::vector<float> partial(blocks);
    cudaMemcpy(partial.data(), d_sums, blocks * sizeof(float), cudaMemcpyDeviceToHost);
    float total = std::accumulate(partial.begin(), partial.end(), 0.0f);
    std::cout << "sum = " << total << "\n";
    cudaFree(d_in); cudaFree(d_sums);
    return 0;
}

```

14. Interview Questions And Model Answers

Q: What happens when you launch kernel<<>>?

A: The host queues work for the GPU. The launch creates a grid of blocks; each block contains threads. The launch is usually asynchronous with respect to the CPU.

Q: Why do we need an $(i < n)$ guard?

A: Grid sizes are commonly rounded up. Extra threads would otherwise access memory past the valid range.

Q: What is a warp?

A: A hardware scheduling group of threads, commonly 32. Threads in a warp execute together, so divergent branches serialize paths.

Q: Shared memory vs global memory?

A: Global memory is large and high latency. Shared memory is per-block, much lower latency, and used for cooperation and data reuse.

Q: What is occupancy?

A: The ratio of active warps on an SM to the hardware maximum. It matters because more resident warps can hide latency, but maximum occupancy is not always maximum performance.

Q: How do you optimize a memory-bound kernel?

A: Reduce bytes moved, coalesce accesses, reuse data through shared memory/cache, avoid unnecessary transfers, and consider fusion.

Q: When are atomics appropriate?

A: When multiple threads update the same location and exact correctness matters. Use them carefully because contention can serialize execution.

Q: How do streams improve performance?

A: They let independent copies and kernels be queued separately so the GPU can overlap transfers and compute when dependencies and hardware permit.

Q: How do you profile CUDA?

A: Start with end-to-end timing, use CUDA events for GPU work, Nsight Systems for timeline gaps/overlap, and Nsight Compute for kernel metrics.

Q: Why use cuBLAS instead of writing GEMM?

A: Vendor libraries are heavily optimized for architecture-specific tiling, tensor cores, memory layouts, and edge cases.

15. 0-to-10 Roadmap

Level	Focus	You can prove it by
0	C++ refresh and toolchain	Compile C++ and CUDA hello-world programs.
1	Kernel launches and indexing	Write vector add and SAXPY from memory.
2	CUDA memory APIs	Explain malloc/copy/free and error checking.
3	Thread/block/grid mapping	Handle 1D and 2D data correctly.
4	Shared memory	Implement tiled matrix multiplication.
5	Reductions and atomics	Implement sum reduction and explain contention.
6	Streams and events	Overlap copy/compute and time kernels correctly.
7	Profiling	Use Nsight to justify a bottleneck.
8	Advanced memory/performance	Discuss coalescing, bank conflicts, occupancy, divergence.

Level	Focus	You can prove it by
9	AI/ML systems	Use cuBLAS/cuDNN or write a PyTorch CUDA extension.
10	Expert interview readiness	Whiteboard kernels, debug failures, and defend optimization choices.

16. Sources And Further Reading

- NVIDIA CUDA C++ Programming Guide: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA CUDA C++ Best Practices Guide: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- NVIDIA CUDA Runtime API: <https://docs.nvidia.com/cuda/cuda-runtime-api/>
- NVIDIA CUDA Installation Guide for Microsoft Windows:
<https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/>
- NVIDIA Nsight Compute User Guide: <https://docs.nvidia.com/nsight-compute/>
- NVIDIA Deep Learning Performance Documentation: <https://docs.nvidia.com/deeplearning/performance/>